Implementación de PPO para Car Racing

Máximo Gubitosi
Universidad de San Andrés
Buenos Aires, Argentina
mgubitosi@udesa.edu.ar

Gabor Gorondi
Universidad de San Andrés
Buenos Aires, Argentina
ggorondi@udesa.edu.ar

***Abstract*—Este trabajo presenta la implementación desde cero y el análisis experimental de un agente basado en *PPO-Clip* (Proximal Policy Optimization con objetivo recortado) para el entorno de carrera *CarRacing-v3*. A partir de observaciones en píxeles y una arquitectura *actor-critic* convolucional, el agente logra recompensas medias superiores a 900 puntos, competitivas con implementaciones de referencia y cercanas al estado del arte.**

Nuestros aportes principales son: (1) una implementación propia de PPO-Clip, validada mediante un estudio de ablación, y comparación con una implementación de *Stable Baselines 3*; (2) una exploración de configuraciones y detalles de arquitectura desde nuestra versión inicial hasta alcanzar resultados cercanos al SOTA; (3) un estudio de reducción de dimensionalidad mediante PCA que muestra que es posible comprimir las observaciones visuales a espacios latentes de tan solo 3–24 dimensiones y entrenar agentes competitivos (700–750 puntos de media) con redes de apenas cientos de parámetros; y (4) un experimento divertido de restringir el espacio de acciones (agentes *Only-Left/Right*) que revelan políticas compensatorias creativas, ilustrando la robustez de PPO-Clip ante un entorno modificado más difícil.

Índice

1 Introduccion 1
1.1 Entorno: Car Racing V3 1
1.2 Implementación de PPO-Clip 2
1.3 Estudio de Ablación y Validación 2
1.4 Arquitectura de Red 2
2 Desarrollo 2
2.1 Investigación y Configuración Inicial 2
2.2 Transición al Espacio Discreto y Primeros Obstáculos . 3
2.3 Comparación con Stable-Baselines3 4
2.4 Estrategia Final: Entrenamiento en Etapas (Curriculum Learning) 5
2.5 Reducción de dimensionalidad mediante PCA 5
2.6 Experimento bonus: agentes con acciones de giro restringidas 9
3 Conclusiones 9
4 Trabajo a Futuro 10
Referencias 10

1 Introduccion

1.1 Entorno: Car Racing V3

El entorno seleccionado para la experimentación es `CarRacing-v3`, de la librería Gymnasium. Se trata de una simulación de conducción de vista superior (top-down) donde el agente controla un automóvil de carreras con el objetivo de visitar la mayor cantidad de segmentos de la pista posibles en el menor tiempo.

En cada episodio del entorno se genera una pista aleatoria compuesta de curvas, rectas y chicanas. Esto impide que el agente memorice una secuencia de acciones fija, obligándolo a aprender a navegar basándose en la percepción visual en tiempo real. El entorno presenta desafíos significativos debido a que el espacio de estados es de alta dimensionalidad y la física del vehículo incluye inercia, fricción y derrapes, lo que requiere un control dinámico preciso.

1.1.1 Espacio de Observación

El entorno devuelve una observación RGB de dimensiones $96 \times 96 \times 3$ píxeles, que representa la vista superior del vehículo y sus alrededores inmediatos. Sin embargo, el render del juego mostrado al usuario se presenta en una mejor calidad de $400 \times 600 \times 3$.

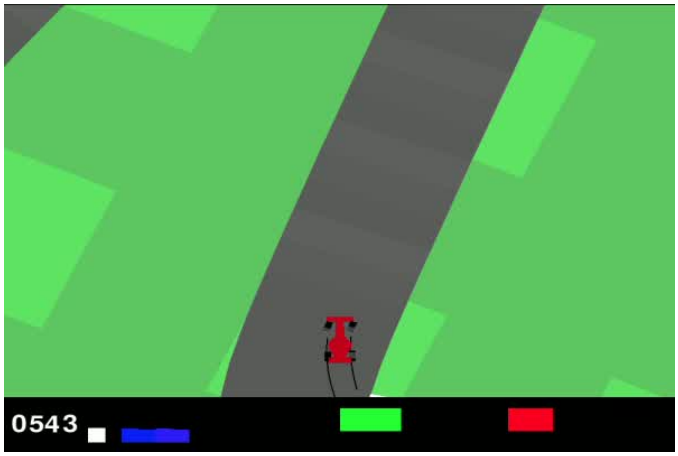

Fig. 1: Ejemplo de observación renderizada ($400 \times 600 \times 3$) con la barra de telemetría inferior. De izquierda a derecha: Velocidad real (barra blanca), Sensores ABS (4 indicadores azules correspondientes a la tracción de cada rueda), Posición del volante (barra verde) y Giroscopio (barra roja), que visualiza la velocidad angular física del vehículo y solo se activa cuando existe rotación efectiva del chasis.

1.1.2 Espacio de Acciones

En la versión del entorno utilizada (v3) se presentan dos espacios de acciones nativos: continuo y discreto. El espacio continuo consiste en un vector de 3 valores flotantes: dirección $[-1, 1]$, aceleración $[0, 1]$ y freno $[0, 1]$. Por otro lado, el espacio discreto (utilizado frecuentemente para simplificar la convergencia en algoritmos como PPO) consiste en 5 acciones

atómicas mutuamente excluyentes: (1) Do nothing, (2) Steer Right, (3) Steer Left, (4) Gas, (5) Brake.

1.1.3 Función de Recompensa y Dinámica

La función de recompensa base está diseñada para fomentar el avance rápido a lo largo de la pista:

- Se otorga una recompensa de -0.1 puntos por cada *frame* transcurrido, penalizando el tiempo y la inacción.
- Se otorgan $+1000/N$ puntos por cada nueva "baldosa" o segmento de la pista visitado (donde N es el número total de baldosas de la pista generada).
- Se otorga una recompensa de -100 si el automóvil llega al límite del mapa

Si el agente lograra recorrer toda la pista perfectamente, obtendría 1000 puntos menos la penalización por tiempo. El episodio finaliza cuando se visitaron todos los segmentos de la pista, si el agente llega al límite del mapa, o después de 1000 pasos (steps/frames).

1.2 Implementación de PPO-Clip

Para abordar el problema de control en CarRacing-v3, seleccionamos el algoritmo Proximal Policy Optimization (PPO) en su variante "Clipped Objective". Esta elección se fundamenta en la capacidad del algoritmo para mantener la estabilidad del entrenamiento en entornos con alta varianza y espacios de observación dimensionalmente grandes (píxeles), evitando las caídas catastróficas de rendimiento comunes en métodos de gradiente de política estándar. Por otro lado PPO-Clip suele ser menos sensible al ajuste de los hiperparámetros y mas simple de implementar desde cero, sin sacrificar robustez.

1.3 Estudio de Ablación y Validación

Con el objetivo de identificar el impacto de cada componente y validar nuestra implementación del algoritmo, diseñamos un estudio de ablación donde se entrena el agente bajo distintas condiciones, contrastando el rendimiento contra nuestra configuración "Baseline". Los resultados, ilustrados en la Figura 2, permiten extraer las siguientes conclusiones:

1) **Baseline:** La configuración completa (Clipping $\epsilon = 0.2$, GAE $\lambda = 0.95$, Entropía, Stack=2) demostró el desempeño más robusto (curva verde), alcanzando rápidamente la estabilidad sobre los 600 puntos. Esto valida la correcta integración de todos los módulos implementados.
2) **No-Clip:** Al desactivar el mecanismo de clipping (curva roja), se observa una volatilidad extrema. Aunque alcanza picos altos momentáneos, sufre caídas catastróficas (colapso de política) debido a actualizaciones de gradiente excesivas, confirmando que el recorte es indispensable para la estabilidad en PPO.
3) **No-Entropy:** Al eliminar el coeficiente de entropía (curva naranja), el agente se estanca cerca de 0 puntos durante más de 1.2M de pasos. La falta de incentivo a la exploración provoca una convergencia prematura a óptimos locales marcados por estrategias pasivas. Esto reduce drásticamente el aprendizaje inicial.

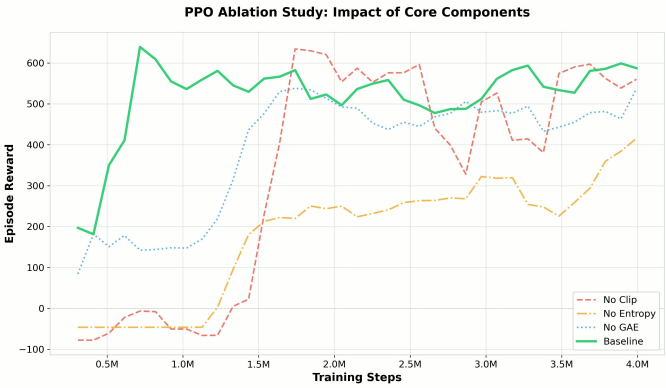

Fig. 2: Curvas de aprendizaje del estudio de ablación (4M pasos). Se observa cómo la configuración completa (Baseline, verde) supera en estabilidad y rendimiento final a las variantes degradadas, confirmando el funcionamiento y la necesidad de cada componente del algoritmo.

4) **No-GAE (Monte Carlo Puro):** Sin el suavizado de GAE (curva azul), el agente logra aprender, pero se mantiene sistemáticamente por debajo del Baseline. La mayor varianza en la estimación de los retornos dificulta el ajuste fino de la política, limitando el rendimiento asintótico final.

1.4 Arquitectura de Red

El agente utiliza una arquitectura *Actor-Critic* con parámetros compartidos. El flujo inicia con una CNN común que procesa el stack de frames ($48 \times 48 \times 2$) para extraer un vector de características de 128 dimensiones. Este vector latente alimenta a dos secciones independientes: el Actor (política) y el Crítico (valor).

TABLE I: Arquitectura del extractor de features

Layer	Configuration	Output Shape	Params
Input	2 frames (Grayscale)	$(2, 48, 48)$	0
Conv2D + ReLU	32 filters, 5×5, s=2	$(32, 22, 22)$	1,632
Conv2D + ReLU	64 filters, 3×3, s=2	$(64, 10, 10)$	18,496
Conv2D + ReLU	128 filters, 3×3, s=2	$(128, 4, 4)$	73,856
Conv2D + ReLU	128 filters, 3×3, s=2	$(128, 1, 1)$	147,584
Flatten	-	$(128,)$	0
Total			**241,568**

1) **El Actor:** Responsable de definir la política π_θ. Se implementaron dos variantes según la etapa de experimentación: una discreta (Tabla II) para el modelo final y una continua (Tabla III) para las pruebas iniciales.
2) **El Crítico:** Estima el valor escalar $V(s)$ del estado para el cálculo de la ventaja (Tabla IV).

2 Desarrollo

2.1 Investigación y Configuración Inicial

El primer paso consistió en analizar implementaciones existentes de PPO en entornos de *CarRacing* que lograban

TABLE II: Actor: Variante Discreta (Final)

Layer	Configuration	Output Shape	Params
Input	Feature vector	$(128,)$	0
Linear + ReLU	$128 \rightarrow 128$ units	$(128,)$	16,512
Linear	$128 \rightarrow 5$ units (logits)	$(5,)$	645
Total			**17,157**

TABLE III: Actor: Variante Continua (Experimental)

Layer	Configuration	Output Shape	Params
Input	Feature vector	$(128,)$	0
Linear + Tanh	$128 \rightarrow 256$ units	$(256,)$	33,024
Linear $(\mu, \log \sigma)$	$256 \rightarrow$ 3+3 units	$(6,)$	1,542
Total			**34,566**

puntajes altos. Identificamos un consenso en ciertas técnicas de preprocesamiento que decidimos adoptar para nuestra implementación desde cero:

- **Recorte y Escala:** Reducir la imagen a 48×48 en escala de grises y cortar la barra inferior de indicadores reduce drásticamente la dimensionalidad de la entrada sin perder información espacial relevante para la conducción. La información de la barra de abajo suele ser demasiado compleja y ademas, en la versión cruda (de menor dimensionalidad que la renderizada), se pierde mucho la calidad de los números y barras, dificultando su comprensión en una CNN.
- **Frame Stacking y Skipping:** Apilar frames permite a la red neuronal inferir velocidad y dirección, mientras que saltar frames (skipping) mejora la eficiencia computacional al no procesar estados intermedios redundantes.

Fig. 3: Comparación entre el estado presentado, el crudo y el preprocesado que ingresa a la red. Las imágenes son de $400 \times 600 \times 3$, $96 \times 96 \times 3$, y $42 \times 48 \times 1$ respectivamente.

Inicialmente, se abordó el problema utilizando el espacio de acciones continuo nativo. En esta configuración, la red neuronal (actor) debe generar una distribución de probabilidad para tres variables acopladas: dirección, aceleración y freno.

A pesar de entrenar durante 15 millones de pasos, los resultados evidenciaron dificultades intrínsecas de esta aproximación. El agente se obtuvo un promedio de recompensa cercano a 500-600 puntos, con una elevada varianza en los puntajes obtenidos en distintos episodios. Se puede observar el rendimiento variado del agente en los videos adjuntos, donde se obtienen puntajes de 360 y 936. Además, como se observa

TABLE IV: Crítico (Red de Valor)

Layer	Configuration	Output Shape	Params
Input	Feature vector	$(128,)$	0
Linear + ReLU	$128 \rightarrow 128$ units	$(128,)$	16,512
Linear	$128 \rightarrow 1$ unit	$(1,)$	129
Total			**16,641**

en la Figura 4, existen caídas drásticas en el rendimiento (por ejemplo, luego de los 8M de pasos), sugiriendo una alta varianza en la estimación de gradientes que impide una convergencia monótona.

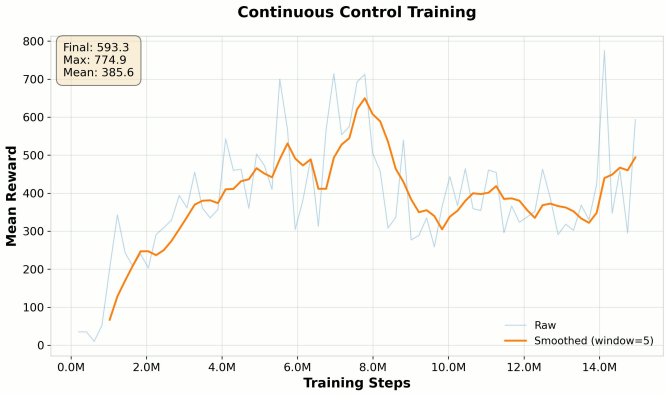

Fig. 4: Curva de entrenamiento utilizando el espacio de acciones continuo durante 15M de pasos. Cada $num_envs \times num_steps \times eval_interval$, (204800 steps en el caso dado), se efectúan 10 episodios enteros de evaluación que definen el reward promedio, correspondiente al eje y del gráfico.

Debido a esta complejidad para aprender simultáneamente la dirección exacta y la aceleración precisa y lograr resultados estables, se decidió pivotar hacia un espacio de acciones discreto.

2.2 Transición al Espacio Discreto y Primeros Obstáculos

Al cambiar al espacio de acciones discreto (5 acciones), notamos una mejora inmediata en la velocidad de aprendizaje, figura 5. La simplificación del espacio de búsqueda permitió al agente estabilizarse más rápido sin sacrificar significativamente el control del vehículo.

Sin embargo, al acercarnos a una recompensa media de 800, volvió a estancarse el progreso de aprendizaje del modelo. En esta instancia detectamos dos problemas centrales que impedían el progreso, los cuales se pueden visualizar claramente en este ejemplo de 622 puntos:

1) **El problema del frenado:** El agente no aprendía a frenar. Esto lo observamos en el tensorboard, donde registramos con qué proporción el modelo toma cada acción. En todos los modelos discretos entrenados hasta el momento se obtuvo un valor constante de 0 con algunos picos momentáneos de hasta 5%. La dificultad

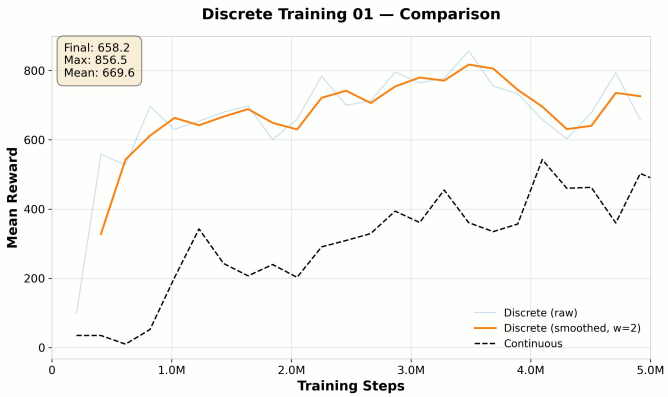

Fig. 5: Entrenamiento usando el espacio de acciones discreto y manteniendo los otros hiperparámetros iguales. La línea punteada muestra el progreso del modelo anterior (con acciones continuas). Se puede observar que el rendimiento con acciones discretas es mejor a lo largo de todo el periodo.

radica en que, en el corto plazo, frenar reduce la recompensa acumulada (debido a la penalización por paso de tiempo y menor distancia recorrida). Sin embargo, es una acción crítica para negociar curvas cerradas. El beneficio de frenar es una **recompensa demorada** (evitar salirse de la pista en el futuro), lo que dificulta que el agente asocie la acción inmediata de "frenar" con la ganancia a largo plazo de mantenerse en pista.

2) **Recuperación errónea:** Al evaluar sin truncamiento, si el auto se salía de pista, a menudo lograba volver pero conduciendo en dirección contraria. Aunque subóptimo, esto le permitía, pasar el punto inicial y explorar nuevas secciones y obtener recompensa positiva, perpetuando un comportamiento indeseado.

Intentamos solucionar esto con *Reward Shaping* (clipping de rewards) y *LR Schedulers*, además de experimentar reduciendo el *Frame Skip* de 4 a 2, pero las mejoras fueron marginales y el agente seguía sin dominar las curvas críticas. El reward shaping implementado consistió en limitar (*clip*) las recompensas positivas a un valor máximo de 1.0, normalizando así la señal de progreso. Esto apunta a contrarrestar el hecho de que el entorno otorga recompensas de magnitud variable ($1000/N$) por visitar nuevos segmentos, por lo que la escala de los retornos puede generar una alta varianza en la estimación de la función de valor. Las penalizaciones negativas se mantuvieron inalteradas (sin *clipping*) para preservar la fuerza de la señal de castigo ante comportamientos indeseados como irse del mapa o no moverse.

2.3 Comparación con Stable-Baselines3

Como referencia externa, comparamos nuestra implementación de PPO-Clip con la implementación de *PPO* de Stable-Baselines3 (SB3), usando `CnnPolicy`. En ambos casos se utilizó el mismo entorno `CarRacing-v3` con los mismos *wrappers* de preprocesamiento (`CarRacingPreprocess`, `FrameStackWrapper`, `OffRoadPenaltyWrapper`), el mismo espacio de acciones discreto, y un conjunto de hiperparámetros alineado (mismos valores de γ, λ, ϵ de clipping, coeficientes de entropía y valor, tamaño de *batch*, número de épocas por actualización, etc.).

Las Figuras 6 y 7 muestran la recompensa media y máxima durante el entrenamiento para ambos métodos.

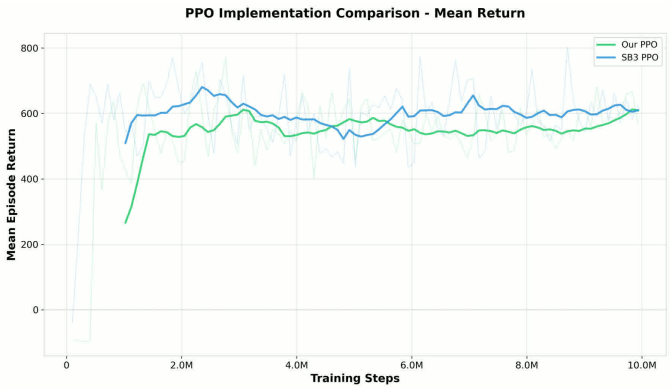

Fig. 6: Evolución del retorno medio durante el entrenamiento para PPO en Stable-Baselines3 (SB3) y nuestra implementación.

Fig. 7: Evolución del retorno máximo observado en las evaluaciones a lo largo del entrenamiento para el PPO de SB3 y nuestra implementación.

SB3 converge un poco más rápido, alcanza recompensas máximas superiores a 800 alrededor de los $\sim$ 300k pasos, mientras que nuestra implementación tarda cerca de $\sim$ 500k pasos en llegar a la misma región. Muestran una varianza similar de rewards medios durante el entrenamiento. En términos de tiempo total de entrenamiento, la corrida con SB3 resultó aproximadamente un 17% más rápida. Este valor es conservador, ya que el tiempo total incluye tareas lentas comunes (exportación de GIFs por ejemplo), por lo que la diferencia real a nivel de algoritmo probablemente sea mayor.

La comparación sugiere que, aunque SB3 es algo más eficiente y estable, nuestra implementación alcanza un rendimiento comparable bajo una configuración de entrenamiento similar.

2.4 Estrategia Final: Entrenamiento en Etapas (Curriculum Learning)

Para superar los obstáculos mencionados, diseñamos una estrategia de entrenamiento progresiva de 30 millones de pasos (steps) dividida en 3 etapas. La hipótesis fue que iniciar con condiciones más fáciles (timeout corto) mejoraría la calidad de los datos en el buffer facilitando el aprendizaje inicial, para luego relajar las restricciones progresivamente y permitir al modelo aprender casos mas complejos, como frenar en una curva pronunciada.

Etapa 1 (0 - 12M steps): Entrenamiento base con *Frame Stack y Skip* de 2 y un max_offroad_seconds muy estricto de 2 segundos. El objetivo es forzar al agente a reiniciar rápidamente si cometía un error, llenando el buffer solo con experiencias de ”buena conducción”.

Etapa 2 (12M - 24M steps): Se cargó el modelo anterior y se relajó el max_offroad_seconds a 7 segundos, pero añadiendo también una penalización (offroad_penalty=5). En esta etapa buscamos permitir al agente aprender a recuperarse de errores menores en curvas, penalizando fuertemente el salir de la pista para incentivar el frenado indirectamente.

Etapa 3 (24M - 30M steps): Se relajó el max_offroad_seconds a 15 segundos, manteniendo la penalización. Esta instancia busca refinar el entrenamiento para lograr robustez general.

Material Complementario: Se pueden visualizar los comportamientos aprendidos en las etapas finales a través de los siguientes videos adjuntos:

- Ejemplo de recuperación tras salida de pista.
- Desempeño del modelo final (SOTA).

Esta estrategia resultó en un modelo capaz de alcanzar recompensas medias de **900** como muestra la figura 8, superando los problemas de frenado (figura 9) y dirección de re-ingreso y logrando un desempeño competitivo con el estado del arte para este entorno. En la figura 10 se puede observar la convergencia de la recompensa máxima obtenida, a los valores mas altos posibles del entorno, dados por $1000 - 0.1 \times frames_{totales}$.

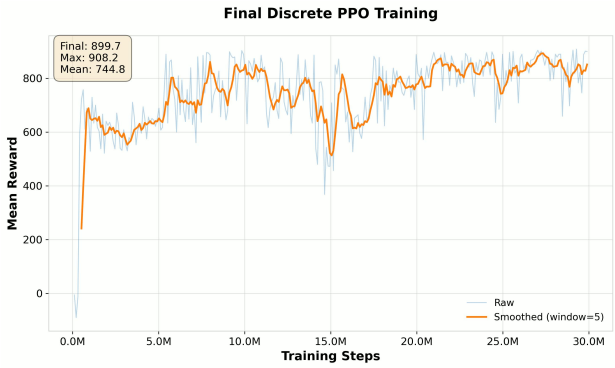

Fig. 8: Curva de recompensa media a lo largo de las 3 etapas de entrenamiento, alcanzando la convergencia en 900 puntos.

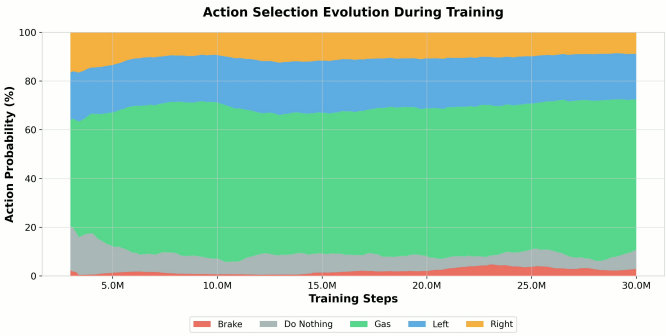

Fig. 9: Evolución de las probabilidades de acción durante el entrenamiento PPO (suavizado con ventana de 100 pasos). El agente rápidamente aprende a favorecer la acción Gas (60%) y a disminuir el uso de ”Do nothing”. Se puede observar que la selección de ”Brake” aumenta recién alrededor de los 15M steps, comprobando la hipótesis de que es una acción más sutil y compleja de aprender a controlar. La diferencia entre el uso de las direcciones se debe a que la pista es siempre un ciclo hacia la izquierda

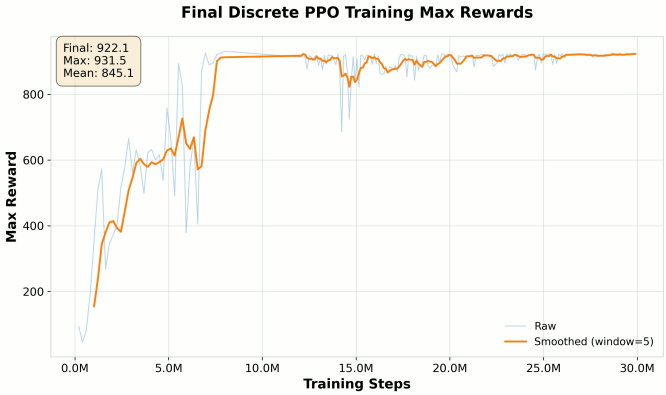

Fig. 10: Recompensa máxima durante el entrenamiento final de PPO. El agente converge a 920 puntos después de 8M de pasos.

2.5 Reducción de dimensionalidad mediante PCA

Además del enfoque estándar basado en una CNN convolucional como feature extractor, exploramos un diseño alternativo en la que el estado visual del entorno se proyecta a un espacio latente de muy baja dimensión mediante *Principal Component Analysis* (PCA), y dicho vector comprimido se utiliza como entrada de una red neuronal diminuta. El objetivo fue estudiar hasta qué punto es posible “condensar” la información relevante de *CarRacing-v3*, y el tamaño de los modelos actor critic, sin destruir la capacidad del agente para aprender una política competente.

La intuición detrás de este enfoque es que, para conducir razonablemente bien, el agente no necesita toda la información contenida en la imagen completa, sino acceder a una descripción de muy baja dimensión que condense la información

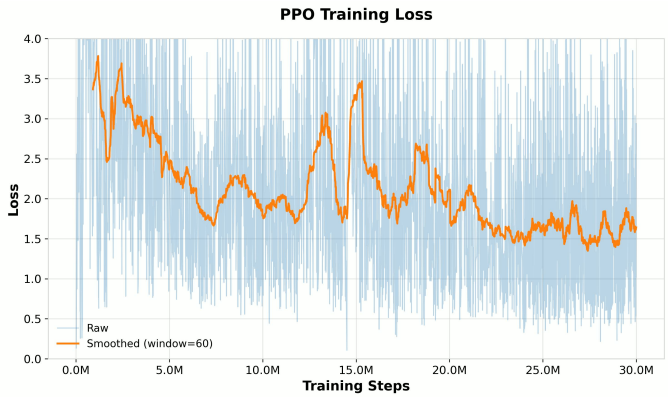

Fig. 11: Evolución del loss total durante el entrenamiento final. Se observa un incremento abrupto a los 12M steps debido al reinicio del learning rate scheduler, seguido por una convergencia más estable en la segunda reanudación (24M) donde el scheduler continuó desde el LR final anterior. La tendencia general muestra reducción del loss de 3.5 a 1.5-1.8

más relevante del ambiente; cosas como: (i) la dirección global de la pista (izquierda/derecha), (ii) la posición relativa del auto respecto al centro del camino, (iii) la curvatura local, y (iv) la curvatura futura. Si tales factores se encontrasen aproximadamente representados en algún espacio latente al que pudiésemos acceder con algún método paramétrico (que después se pueda aplicar a nuevas imágenes no vistas), entonces la tarea de control podría resolverse con un modelo prácticamente lineal sobre esas representaciones.

En este contexto, PCA (*Principal Component Analysis*) resulta una herramienta viable para nuestro propósito. Es un método de reducción de dimensionalidad simple y eficiente, que busca una nueva base ortogonal de vectores (componentes principales), mediante descomposición en valores singulares, tal que las primeras componentes expliquen la mayor parte posible de la varianza de los datos. Intuitivamente, cada componente captura una ”dirección de variación típica” en el espacio de imágenes, lo cual podría traducirse a lograr capturar la inclinación y curva de la pista por ejemplo. Al proyectar cada observación sobre unas pocas componentes, obtenemos un vector latente de baja dimensión que preserva la estructura dominante del entorno y es potencialmente suficiente para que el agente aprenda una buena política de control.

2.5.1 Construcción del dataset y preprocesamiento

Para ajustar el modelo de PCA recolectamos un conjunto de datos estático de aproximadamente $50{,}000$ observaciones a partir de episodios jugados manualmente. Para este entorno, nos pareció razonable entrenar un PCA con este conjunto estático de samples que tiene buena representatividad del entorno, y luego dejar el modelo entrenado fijo durante el entrenamiento del PPO para garantizar la estabilidad del espacio latente.

Luego, a fines de maximizar la efectividad de la reducción de dimensionalidad, las observaciones originales del entorno ($96 \times 96 \times 3$) se transformaron mediante la siguiente *pipeline*:

- Conversión a escala de grises, resize, y recorte de la barra inferior de indicadores, quedando un área útil de 48×42.
- Binarización por umbral (*clipped black & white*) que diferencia fuertemente entre pista y césped, de manera que la mayor parte de la varianza visual se concentre en la geometría del camino.
- En una segunda variante más agresiva, se añadió un paso de *downsampling* adicional a 12×10 antes de aplicar PCA, con el objetivo de analizar el límite de compresión posible. En este caso, además de achicar el modelo downstream el enfoque es reducir también la complejidad computacional del PCA en sí.

A partir de estas imágenes procesadas entrenamos un PCA incremental (IncrementalPCA de scikit-learn) y nos quedamos con varios tamaños del subespacio latente,

$$k \in \{3, 5, 8, 12, 24\},$$

obteniendo así cinco modelos lineales distintos de reducción de dimensionalidad. Para cada valor de k medimos la proporción de varianza total explicada (suma de los *explained variance ratios* de las k primeras componentes). La Figura 12 muestra cómo la varianza explicada crece con el número de componentes, evidenciando que con pocas dimensiones ya se retiene una fracción considerable de la información del entorno.

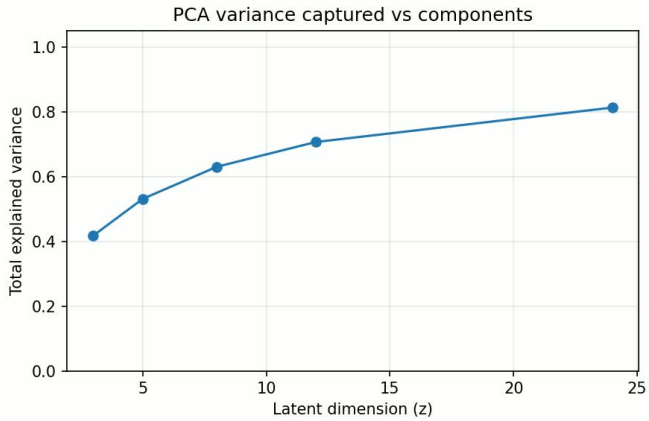

Fig. 12: Porcentaje de varianza total explicada por el PCA en función del número de componentes k. Cada punto corresponde a la suma acumulada de la varianza explicada por las primeras k componentes principales.

La Figura 13 muestra un ejemplo de proyección bidimensional de un sampleo del dataset (tomando las dos primeras componentes de un PCA de dimensión 3). Podemos notar que los datos están distribuidos homogéneamente, sin agrupaciones demasiado densas.

La Figura 14 ilustra, para el caso $k = 5$, reconstrucciones de imágen a partir de los vectores latentes. Se muestra el efecto de variar individualmente cada componente principal

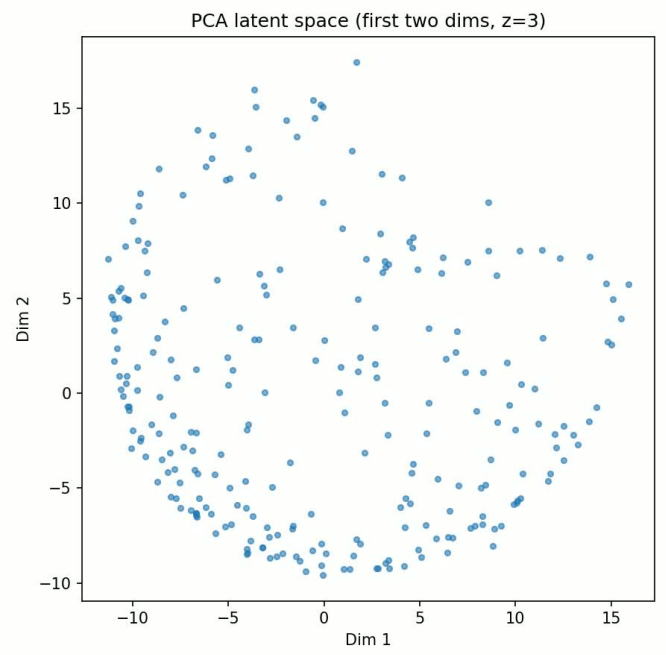

Fig. 13: Distribución de las observaciones en el espacio latente bidimensional (primeras dos componentes principales). Las regiones locales agrupan estados visualmente similares del entorno.

manteniendo las demás fijas. Se observa que las componentes capturan información relevante y semántica de la pista: las primeras dos codifican variaciones en la posición relativa e inclinación hacia la izquierda/derecha, mientras que las componentes 3 a 5 parecen condensar información sobre la curvatura.

2.5.2 Calidad de reconstrucción

En las Figuras 15 y 16 comparamos imágenes originales (preprocesadas a 48×42) con sus reconstrucciones a partir de un PCA de $k = 3$ y de $k = 24$ componentes. Con sólo 3 dimensiones la reconstrucción resulta borrosa y pierde detalles finos, pero preserva la estructura global de la pista (rectas vs curvas, orientación aproximada). Con 24 componentes la reconstrucción es visiblemente cercana a la imagen original, mostrando que el entorno es altamente redundante y que la información relevante puede comprimirse de forma agresiva con métodos lineales.

Resultados análogos se obtuvieron repitiendo el experimento con la versión reducida a 12×10 píxeles. La Figura 17 muestra reconstrucciones para $k = 24$ en esta resolución extremadamente compacta: aunque la imagen es más tosca, la topología local de la pista sigue siendo claramente distinguible.

2.5.3 Arquitectura del agente en el espacio latente

Una vez entrenado el PCA, lo integramos como parte fija del modelo del agente. En cada paso de tiempo:

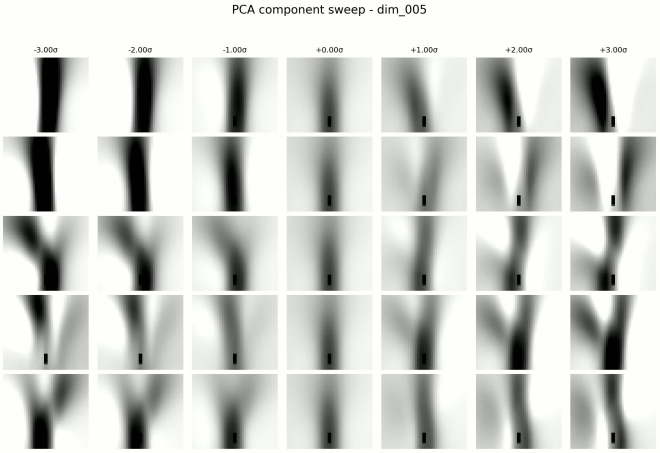

Fig. 14: Reconstrucciones de los *frames* (proyección al espacio original) a partir de los vectores latentes, realizando un barrido de las 5 primeras componentes principales sobre las imágenes binarizadas. Cada fila corresponde a una componente; al variar su valor entre -3 y 3 desviaciones estándar, se observan cambios en la orientación y curvatura de la pista, lo que sugiere que el PCA captura directamente factores semánticos relevantes para el control.

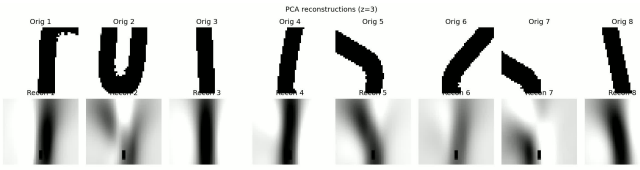

Fig. 15: Reconstrucciones con PCA de dimensión 3 sobre imágenes binarizadas de 48×42. Arriba: frames originales preprocesados. Abajo: reconstrucciones a partir del vector latente de 3 dimensiones.

1) Se aplica el mismo preprocesamiento que en la fase de entrenamiento del PCA (binarización y, opcionalmente, reducción a 12×10).
2) La imagen resultante se proyecta mediante el PCA a un vector latente $z_t \in \mathbb{R}^k$.
3) Para incorporar información dinámica, se apilan las proyecciones de 2 frames, formando un vector de entrada de dimensión $2k$.

En las Tablas V, VI y VII se detalla la arquitectura.

TABLE V: Extractor de features basado en PCA (caso 48×48)

Layer	**Configuration**	**Output Shape**	**Params**
Input	2 frames (BW, preprocesados)	$(2, 48, 48)$	0
Preprocesamiento	Recorte + binarización	$(2, 48, 42)$	0
Flatten	Por frame	$(2, 2016)$	0
PCA por frame	Proyección $2016 \to k$	$(2, k)$	0
Stack temporal	2 frames apilados	$(2k,)$	0
Total			**0**[1]

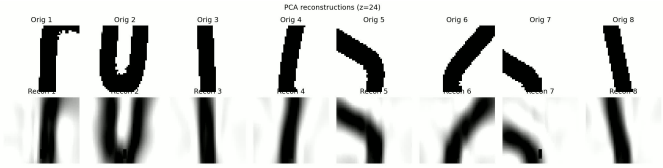

Fig. 16: Reconstrucciones con PCA de dimensión 24 sobre la misma colección de imágenes de 48×42. La estructura de la pista y el contorno del camino se preservan casi por completo.

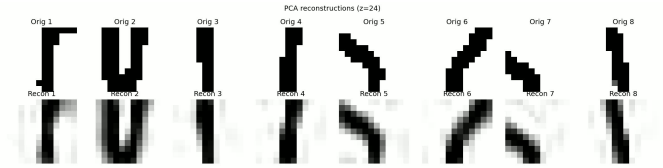

Fig. 17: Reconstrucciones con PCA de dimensión 24 entrenado sobre imágenes reducidas a 12×10. A pesar de la baja resolución, la forma de la pista se preserva suficientemente para la navegación.

TABLE VI: Actor: Variante discreta sobre espacio latente (ejemplo con $k = 5$)

Layer	Configuration	Output Shape	Params
Input	Feature vector (2 frames)	$(10,)$	0
Linear + ReLU	$10 \to 12$ units	$(12,)$	132
Linear	$12 \to 5$ units (logits)	$(5,)$	65
Total			**197**

TABLE VII: Crítico sobre espacio latente ($k = 5$)

Layer	Configuration	Output Shape	Params
Input	Feature vector (2 frames)	$(10,)$	0
Linear + ReLU	$10 \to 12$ units	$(12,)$	132
Linear	$12 \to 1$ unit	$(1,)$	13
Total			**145**

Incluso para el mayor tamaño considerado ($k = 24$, entrada de 48 dimensiones), el número total de parámetros del Actor-Critic sigue siendo del orden de unos pocos cientos, en contraste con los cientos de miles de parámetros de la CNN original (Tabla I). Esto se traduce en una cantidad de operaciones ejecutadasen tiempo de inferencia y entrenamiento varios órdenes de magnitud menor, y como veremos a continuación, se logra mantener un rendimiento competitivo en términos de recompensa media alcanzada, con considerablemente menos pasos de entrenamiento.

2.5.4 Resultados de aprendizaje con PPO

Entrenamos un agente PPO-Clip independiente para cada dimensión latente $k \in \{3, 5, 8, 12, 24\}$, manteniendo una configuración similar de hiperparámetros que en el modelo con CNN. La Figura 18 muestra la recompensa media en función de los pasos de entrenamiento para la variante basada en imágenes de 48×42, mientras que la Figura 19 resume los resultados para la versión de 12×10.

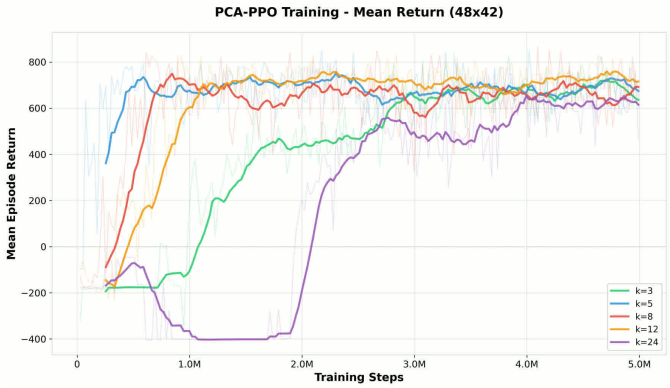

Fig. 18: Curvas de recompensa media para PPO con estados comprimidos mediante PCA entrenado sobre imágenes de 48×42. Cada curva corresponde a un valor distinto de k.

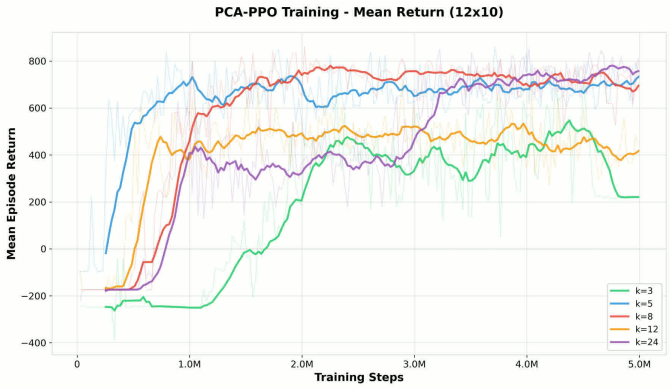

Fig. 19: Resultados análogos para la variante de PCA entrenada sobre imágenes reducidas a 12×10.

Las observaciones principales son:

- Para todos los $k \in \{3, 5, 8, 12, 24\}$ en el caso de imágenes de 48×42 las curvas de aprendizaje convergen a resultados similares. Obtienen recompensas medias en el rango de **700–750** puntos. Esto se sitúa un poco por debajo del modelo con CNN (que alcanza 800–900 de media), pero con un costo computacional enormemente menor. Además acá hicimos menos exploración de hiperparámetros óptimos, y menos steps de entrenamiento.
- En general, a menor k, más rápida la convergencia, aunque esto puede depender del learning rate y otros factores también, dada la diferencia de parámetros del modelo a entrenar en cada caso. El caso más extremo ($k = 3$) presenta cierta inestabilidad.
- La variante entrenada sobre imágenes 12×10 sufre una ligera degradación adicional en el rendimiento asintótico en general, y algunas configuraciones se estancan más facilmente en retornos medianos de ~ 400, pero otras sí llegan a converger similarmente.

[1]El PCA se entrena offline y se mantiene fijo durante el entrenamiento de PPO, por lo que no introduce parámetros actualizables en la fase de RL.

Material Complementario: Se pueden visualizar los comportamientos aprendidos y las reconstrucciones del espacio latente de dos agentes con distinto k:

- $k = 5$ con imágenes de 48×42.
- $k = 24$ con imágenes de 48×42.

De forma cualitativa, los videos adjuntos muestran que el agente basado en PCA reproduce comportamientos de conducción razonables: sigue la pista, toma curvas suaves y evita salirse de la ruta, aunque con más errores en curvas muy cerradas que la versión con CNN, y todavía no llega a aprender a frenar en curvas difíciles o cuando están yendo muy rápido. En conjunto, estos experimentos sugieren que *CarRacing-v3* admite una representación de estado sorprendentemente compacta y que, una vez obtenida, un agente PPO con una red diminuta es suficiente para alcanzar un rendimiento competitivo.

2.6 Experimento bonus: agentes con acciones de giro restringidas

Como experimento bonus exploramos qué tan robusto es PPO-Clip frente a restricciones extremas del espacio de acciones. En particular, entrenamos dos agentes distintos sobre el mismo entorno CarRacing-v3, pero modificando el conjunto de acciones discretas disponibles:

- **Agente *Only-Left***: se elimina la acción de girar a la derecha; el agente sólo puede girar a la izquierda, acelerar, frenar o no hacer nada.
- **Agente *Only-Right***: se elimina la acción de girar a la izquierda.

El entorno, la señal de recompensa y el resto de la configuración de PPO (red CNN + actor discreto, hiperparámetros) se mantuvieron similares al experimento principal. El objetivo fue responder a la pregunta: *¿hasta qué punto puede compensar nuestro PPO restricciones en el espacio de acciones aprendiendo estrategias "indirectas"?*

2.6.1 Curvas de entrenamiento

La Figura 20 muestra la evolución del retorno medio durante el entrenamiento de ambos agentes restringidos. A pesar de la reducción significativa del espacio de acciones, ambos logran aprender políticas con recompensas positivas y estables (algo por debajo del agente completo).

2.6.2 Comportamientos emergentes

Los comportamientos aprendidos por ambos agentes resultan particularmente interesantes e ilustrativos de la flexibilidad de PPO.

Material Complementario: Se pueden visualizar los comportamientos aprendidos por ambos agentes en los siguientes videos adjuntos:

- Only-Left: saltea porciones del mapa.
- Only-Right: derrape controlado hacia el lado contrario.

- **Agente *Only-Left*:** dado que la pista está globalmente "encerrada" hacia la izquierda, el agente descubre una estrategia simple pero efectiva: en curvas hacia la derecha, en lugar de intentar seguir la trayectoria ideal, continúa en línea recta hasta que la pista vuelve a curvar hacia la izquierda, donde puede corregir su trayectoria. En la Figura 21 se observa un ejemplo donde el auto desestima completamente una curva hacia la derecha. Incluso, en el video se ve un caso donde después de dar una vuelta entera al mapa, dobla hacia la izquierda en una curva derecha y de tal forma llega a una sección con *tiles* no vistos anteriormente. Tambien es interesante notar que este agente toma acciones de giro mas medidas y sutiles que el agente original, ejecuta giros mas leves para no pasarse de estar paralelo con la pista.
- **Agente *Only-Right*:** este caso es todavía más interesante. Al no poder girar a la izquierda, el agente aprende a "simular" un giro hacia ese lado realizando una maniobra: antes de una curva pronunciada hacia la izquierda, se sale deliberadamente de la pista, ejecuta generalmente dos giros cerrados hacia la derecha, y reingresa a la pista con una orientación efectiva equivalente a haber girado a la izquierda. La Figura 22 muestra una captura de *frame* donde se observa este patrón de comportamiento.

Fig. 20: Retorno medio durante el entrenamiento para los agentes *Only-Left* y *Only-Right*. Ambos alcanzan políticas razonablemente competentes a pesar de la restricción en el espacio de acciones.

En resumen, estos resultados muestran que PPO-Clip es capaz de aprender políticas menos lineales y exploratorias frente a un ambiente modificado más complejo.

3 Conclusiones

Se logró implementar y entrenar exitosamente un agente basado en PPO-Clip capaz de resolver el entorno *CarRacing-v3* con un desempeño de nivel cercano al máximo alcanzable del entorno (recompensas medias de 900 puntos). El desarrollo completo permitió una comprensión profunda del algoritmo y sus particularidades. Logramos mediante una búsqueda manual de cambios de arquitectura e hiperparámetros incrementar el performance del agente de valores medianos de recompensa promedio a converger cerca del máximo del ambiente. Logramos que el agente aprendiera primero a mantenerse en el camino y posteriormente refinara su técnica para trazar curvas complejas sin salirse del camino.

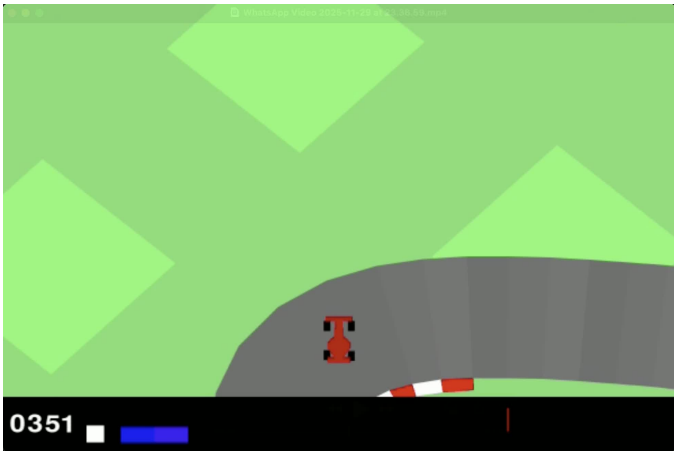

Fig. 21: Agente *Only-Left* en una curva hacia la derecha: al no disponer de la acción de girar a la derecha, tiende a seguir recto hasta que la pista vuelve a curvar hacia la izquierda, donde puede corregir su trayectoria.

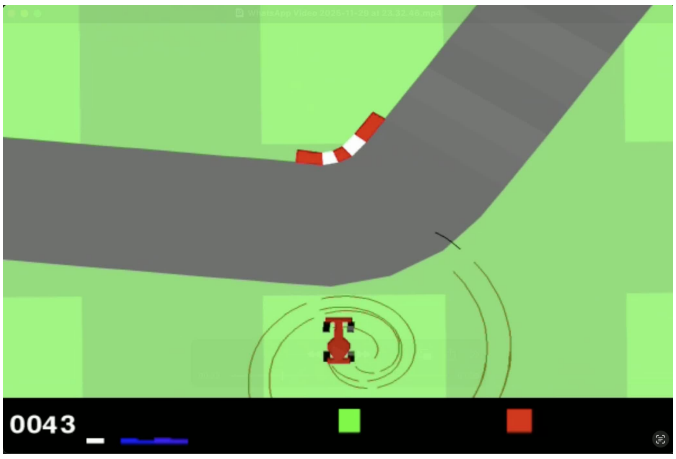

Fig. 22: Agente *Only-Right* realizando un *donut* fuera de la pista para poder "simular" un giro hacia la izquierda. Este patrón se repite de forma consistente en curvas hacia la izquierda.

Asimismo, los experimentos complementarios aportaron un análisis interesante de técnicas para reducir la complejidad computacional de la solución, reducción de dimensionalidad y análisis de espacio latente, relevantes a casos de uso más reales de aprendizaje reforzado en sistemas y ambientes más complejos, donde optimizar el costo computacional se vuelve más clave. La experimentación con PCA demostró que la complejidad visual del entorno es altamente reducible, y así también el tamaño del modelo del agente.

4 Trabajo a Futuro

Como trabajo futuro, se propone explorar la estrategia de Curriculum learning al espacio de acciones continuo, y aplicando este tipo de entrenamiento más largo a los agentes que usaron reducción de dimensionalidad, para ver hasta dónde se puede llegar con éste tipo de enfoque. Además notamos un error en el uso del lr scheduler posterior al entrenamiento de los modelos continuos que seguramente afecto negativamente a la variabilidad en su entrenamiento.

También, sería valioso hacer un análisis del espacio latente aprendido por el modelo convolucional, y compararlo con el espacio latente del PCA, para analizar similaridades y diferencias. En esta línea, también sería muy interesante explorar métodos alternativos de reducción de dimensionalidad como autoencoders variacionales, beta-VAEs, o transformers, para comparar sus rendimientos, ya que al ser métodos no lineales y poderosos podrían aprender representaciones más robustas y/o de menor dimensionalidad y con componentes más desacoplados y descriptivos de la semántica del entorno. Asimismo contrastar el trade-off de eficiencia computacional.

Referencias

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] Farama Foundation, "Car Racing - Gymnasium Documentation," *https://gymnasium.farama.org/environments/box2d/car_racing/*, Accessed: Nov. 2025.

[3] M. Peat, "Solving CarRacing with PPO," *https://notanymike.github.io/Solving-CarRacing/*, 2020.

[4] S. Elsheikh, "Car Racing PPO implementation," *GitHub Repository: https://github.com/elsheikh21/car-racing-ppo*, Accessed: Nov. 2025.